

Lab 1 Report

Machine Learning for Hardware Design

Authors: Zach Dixon, Jongwoo Kim

[Dataset Inspection]

We are required to take 3 images from the dataset to use as test images, printing out some data about the image. This can easily be done by grabbing the images datatype, dimensional shape (depth), and by computing the size of the image.

Export each of the 3 images to binary files so that we can use them in future labs.

```
> v
# TODO: Print and visualize three inputs from the validation set
#      : Print the storage data type
#      : Print and note the dimensions of each image
#      : Print the memory required to store each image

# Sample Images
sample_imgs = []
for index, img_data in enumerate(ds_val.take(3)):
    sample_imgs.append(img_data)
    image, label, id, label_name = img_data["image"], img_data["label"], img_data["id"], img_data["metadata"]['label_name']

    print(f'\n--- Image {index} ---')
    # TODO: Your Code Here
    img_info(img_data)

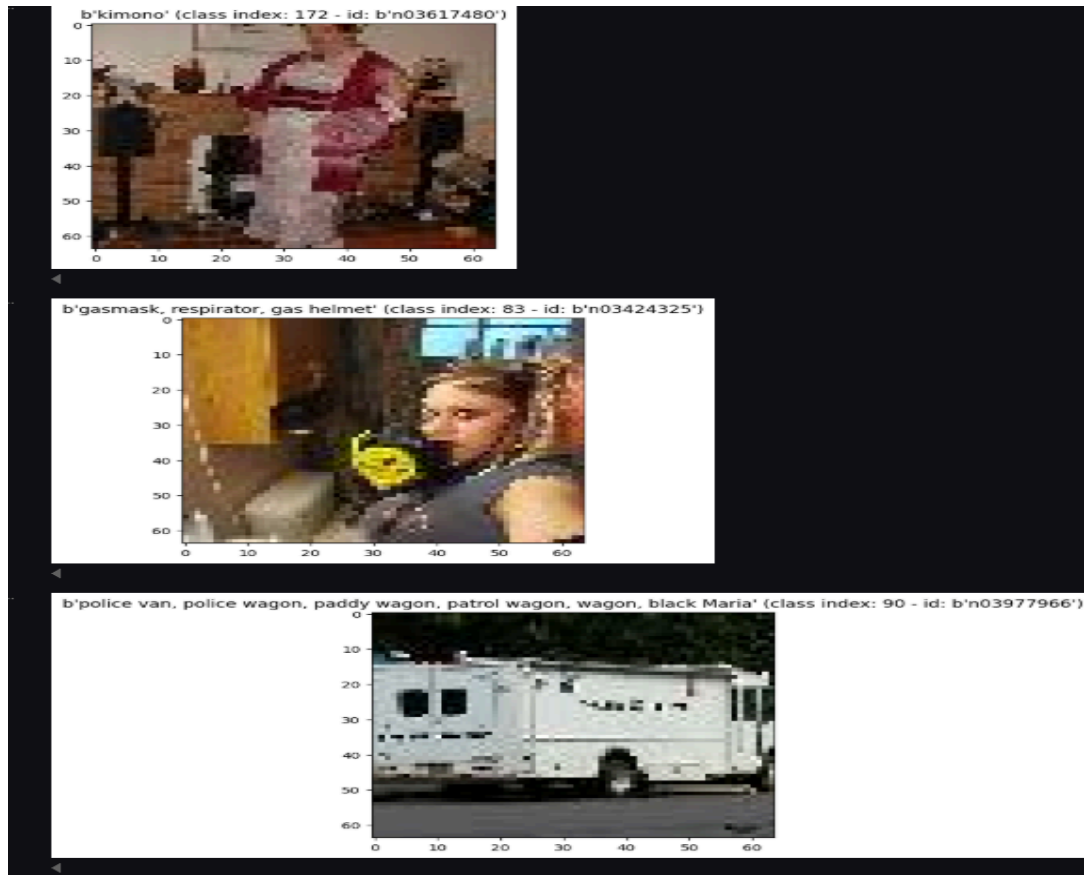
    print(f'Data type: {image.dtype}')
    print(f'Dimensions: {image.shape}')
    print(f'Memory required: {image.numpy().nbytes} bytes')
    # See example usage: https://github.com/duweisu/tiny-imagenet-tfds

--- Image 0 ---
Label: b'kimono' (class index: 172 - id: b'n03617480')
Data type: <dtype: 'uint8'>
Dimensions: (64, 64, 3)
Memory required: 12288 bytes

--- Image 1 ---
Label: b'gasmask, respirator, gas helmet' (class index: 83 - id: b'n03424325')
Data type: <dtype: 'uint8'>
Dimensions: (64, 64, 3)
Memory required: 12288 bytes

--- Image 2 ---
Label: b'police van, police wagon, paddy wagon, patrol wagon, wagon, black Maria' (class index: 90 - id: b'n03977966')
Data type: <dtype: 'uint8'>
Dimensions: (64, 64, 3)
Memory required: 12288 bytes
```

Console output for image class, data type, dimensional array structure, and size.



This is the remaining output of the three selected images, which was too long to fit in a single screenshot.

To save the images for later use, each TensorFlow image was converted to a NumPy array and flattened using `numpy().flatten()`. The flattened image data was then exported to a binary file. The corresponding class and metadata information were saved separately so that the three images can be identified and reused in future labs.

[Model Loading and Inference]

Load the CNN_-tinyimagenet.h5 model using Tensorflow and print the computational graph summary

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 60, 60, 32)	2,432
conv2d_1 (Conv2D)	(None, 56, 56, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 28, 28, 32)	0
conv2d_2 (Conv2D)	(None, 26, 26, 64)	18,496
conv2d_3 (Conv2D)	(None, 24, 24, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_4 (Conv2D)	(None, 10, 10, 64)	36,928
conv2d_5 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544
dense_1 (Dense)	(None, 200)	51,400

Total params: 770,218 (2.94 MB)

Trainable params: 770,216 (2.94 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

Computational Graph Summary

```
Image 0
Best Guess: ['kimono']
Confidence: 0.1276780664920807
Actual Class: b'kimono' (class index: 172 - id: b'n03617480')

Image 1
Best Guess: ['gasmask', 'respirator', 'gas helmet']
Confidence: 0.22464795410633087
Actual Class: b'gasmask, respirator, gas helmet' (class index: 83 - id: b'n03424325')

Image 2
Best Guess: ['moving van']
Confidence: 0.6571351885795593
Actual Class: b'police van, police wagon, paddy wagon, patrol wagon, wagon, black Maria' (class index: 90 - id: b'n03977966')
```

Use 3 model to predict the 3 sample images previously explored

Calculate the Top-1, Top-5, and Top-10 accuracy of the model over the validation dataset

```
# Running inference on our model
# We can run an inference of our model by doing the following (we are doing batches of 1 here)
for example in ds_train.batch(1).take(1):
    img_info(example)

    # Make a prediction
    pred = model.predict(tf.cast(example["image"], tf.float32)/255.0)
    # print(f'Raw 200 Class Weighted Prediction:\n{pred}') # Uncomment to see the raw prediction

    # What is our best guess?
    best_guess = tf.math.argmax(pred, axis=1).numpy() # Our output is 200 weighted value, we want the most likely
    print(f'Best Guess [class index]: {class_names[best_guess[0]]} [{best_guess[0]}]')
    print(f'Best Guess Confidence (percent / 1.0): {pred[0][best_guess]}')

    # What are our top 15 guesses?
    top_15 = tf.math.top_k(pred, k=15)
    print(f'Top 15 Guesses (class index): {[f"{class_names[idx]} [{idx}]" for idx in top_15.indices[0]})\n')
    print(f'Top 15 Guesses Confidence (percent / 1.0): {top_15.values}')

2026-09-05 16:45:40.441545: W tensorflow/core/kernels/data/cache_dataset_ops.cc:858] The calling iterator did not fully read the dataset being cached. In o
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1788644740.548334 126119 service.cc:145] XLA service 0x7fa12c004b10 initialized for platform CUDA (this does not guarantee that XLA will be us
I0000 00:00:1788644740.548356 126119 service.cc:153] StreamExecutor device (0): NVIDIA T1000, Compute Capability 7.5
2026-09-05 16:45:40.593704: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:268] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REP
Label: [b'banana'] (class index: [46] - id: [b'n07753592'])
2026-09-05 16:45:40.735915: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:465] Loaded cuDNN version 8907
1/1 ----- 2s 2s/step
Best Guess [class index]: ['banana'] [46]
Best Guess Confidence (percent / 1.0): [0.27655593]
Top 15 Guesses (class index): ["['banana'] [46]", "['ice cream', 'icecream'] [28]", "['bell pepper'] [188]", "['mantis', 'mantid'] [51]", "['pomegranate']
Top 15 Guesses Confidence (percent / 1.0): [[0.27655593 0.2510916 0.1987927 0.05468671 0.0346917 0.02991656
0.01525334 0.01479298 0.01337347 0.01190735 0.00819934 0.00766431
0.00634047 0.00557536 0.00488306]]
I0000 00:00:1788644742.186049 126119 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.
2026-09-05 16:45:42.258762: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

Here we have found our best guess for the image, as well as the Top-1, Top-5, and Top-10 accuracy of our model over the validation set.

```
Total Images: 10000
Top-1 Accuracy: 24.30%
Top-5 Accuracy: 49.40%
Top-10 Accuracy: 61.75%
```

Model Accuracy for 10,000 images

For top-1 accuracy, we have a very poor 24.3%, 49.4% for top-5 and 61.7% for top-10. I would rate this model 2/5, given it can actually identify aspects of the image. As for its accuracy, it is very poor and does not improve to 50% until we hit 10 guesses (interpolation may yield a smaller number of guesses). Due to our low accuracy at even top-10, I would not use our model. We cannot even hit 50% at top-5 (for this set of images)

```
All Classes:
0: ['Egyptian cat']
1: ['reel']
2: ['volleyball']
3: ['rocking chair', 'rocker']
4: ['lemon']
5: ['bullfrog', 'Rana catesbeiana']
6: ['basketball']
7: ['cliff', 'drop', 'drop-off']
8: ['espresso']
9: ['plunger', 'plumber's helper']
10: ['parking meter']
11: ['German shepherd', 'German shepherd dog', 'German police dog', 'alsatian']
12: ['dining table', 'board']
13: ['monarch', 'monarch butterfly', 'milkweed butterfly', 'Danaus plexippus']
14: ['brown bear', 'bruin', 'Ursus arctos']
15: ['school bus']
16: ['pizza', 'pizza pie']
17: ['guinea pig', 'Cavia cobaya']
18: ['umbrella']
19: ['organ', 'pipe organ']
20: ['oboe', 'hautboy', 'hautbois']
21: ['maypole']
22: ['goldfish', 'Carassius auratus']
23: ['potpie']
...
Number of Classes: 200
Training Samples: 100000
Validation Samples: 10000
Total Samples: 110000
```

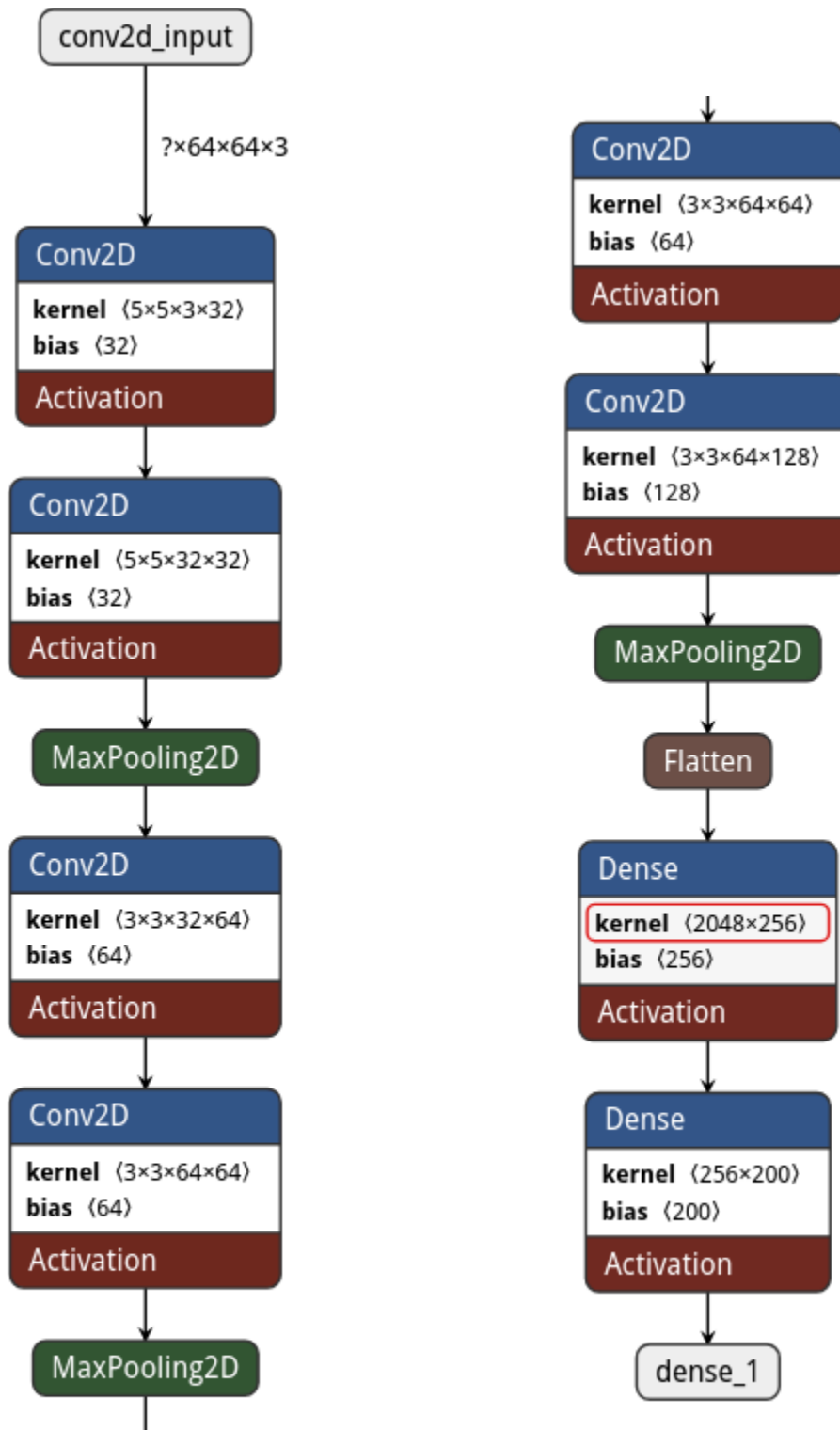
Classes, training and validation samples.

Use the model to predict the 3 sample images you previously explored and report the best guess, associated confidence, and actual class in a table

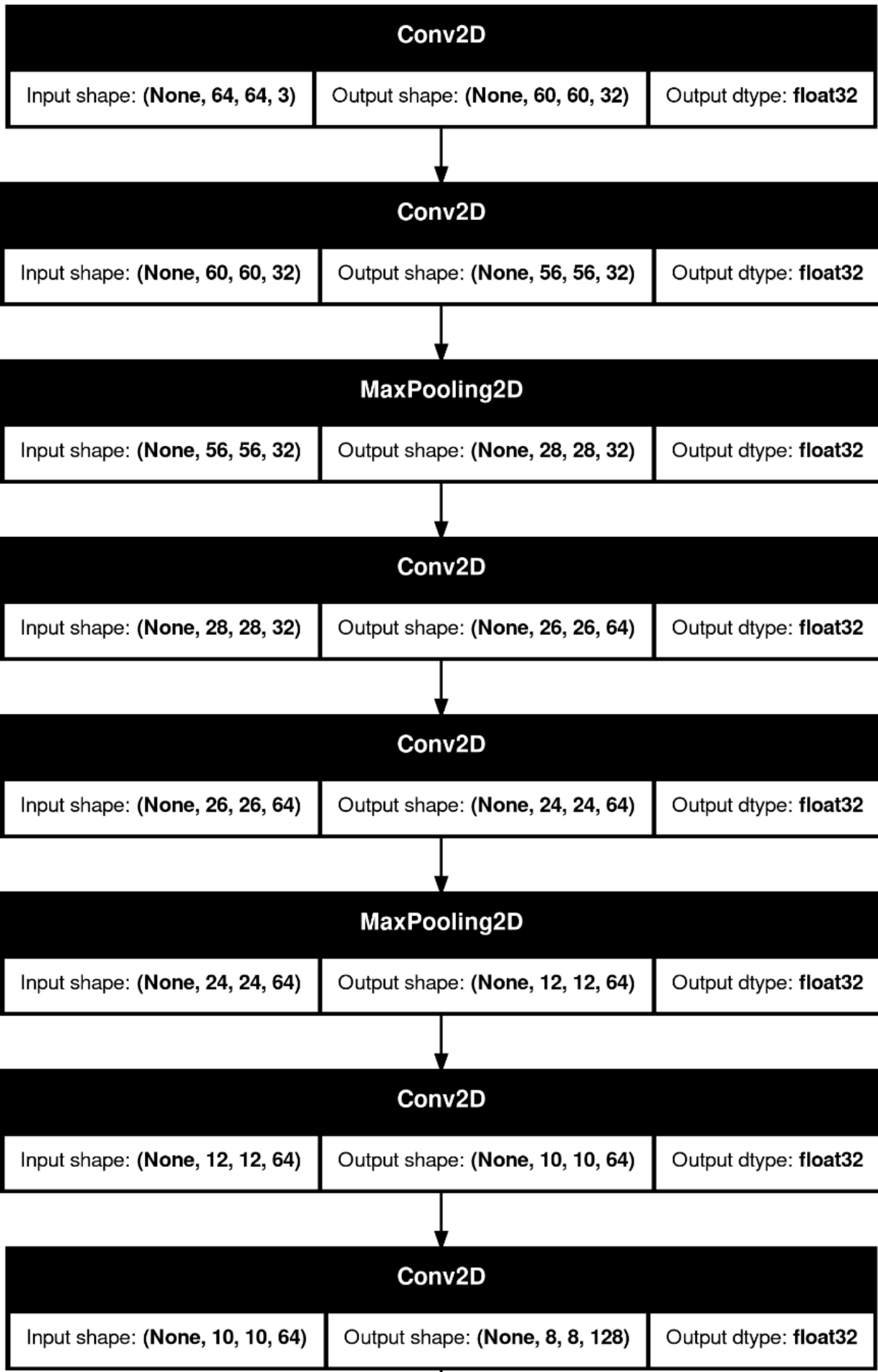
In total, we have 110,000 images, with 100,000 training images and 10,000 validation images across 200 classes. Compared to the original ImageNet dataset, our dataset is much smaller. ImageNet has 1,000 object classes and approximately 1.28 million training images, which is more than 10 times the number of training images in our dataset. The full ImageNet database contains more than 14 million images.

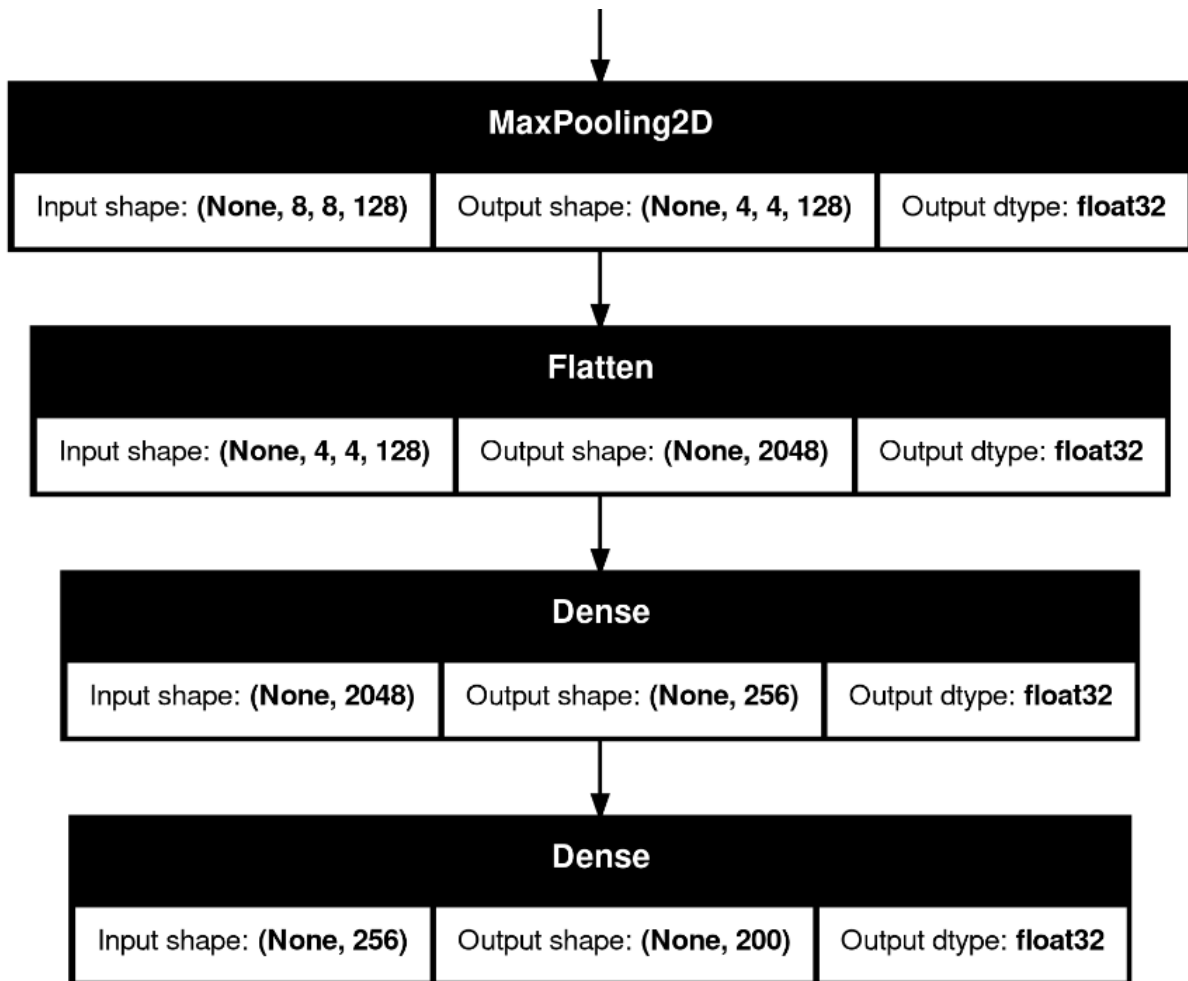
Even though our dataset is smaller, 110,000 images is still a large amount of data. Having many images across 200 classes gives the model many different examples to learn from and can help it generalize when making predictions on new images.

[Model Exploration]



Neutron Visualization

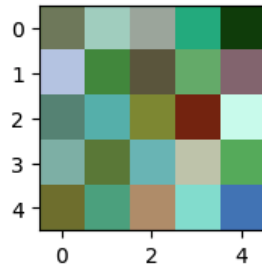




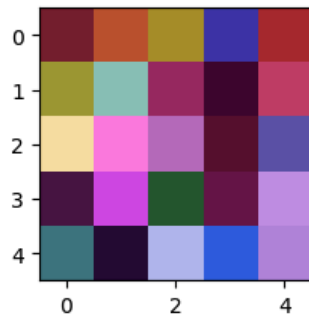
Weight Visualization

```
=== Near Input Layer ===  
Layer:conv2d  
Kernel data type:float32  
Kernel dimensions:(5, 5, 3, 32)  
Kernel Memory:9600 bytes  
Bias Dimensions:(32,)  
Bias Memory:128 bytes  
Total weight memory:9728 bytes
```

Weights of conv2d



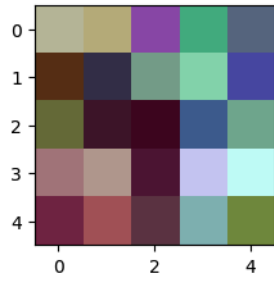
Weights of conv2d



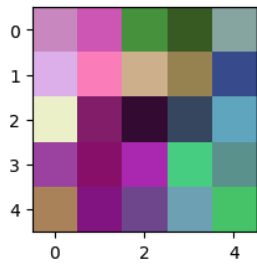
Weights of conv2d

=

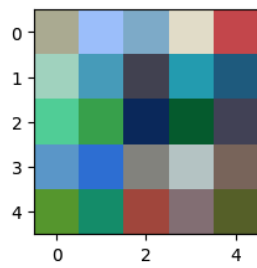
Weights of conv2d



Weights of conv2d

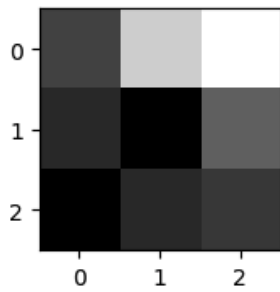


Weights of conv2d

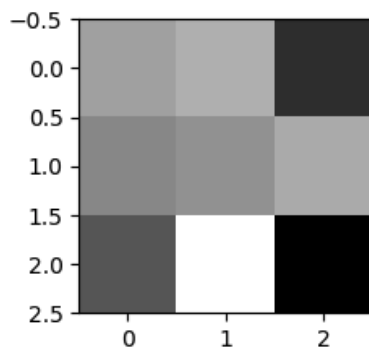


```
=== Near Output Layer ===  
Layer:conv2d_5  
Kernel data type:float32  
Kernel dimensions:(3, 3, 64, 128)  
Kernel Memory:294912 bytes  
Bias Dimensions:(128,)  
Bias Memory:512 bytes  
Total weight memory:295424 bytes
```

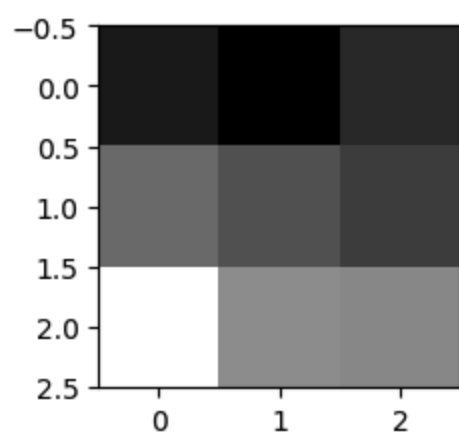
Weights of conv2d_5



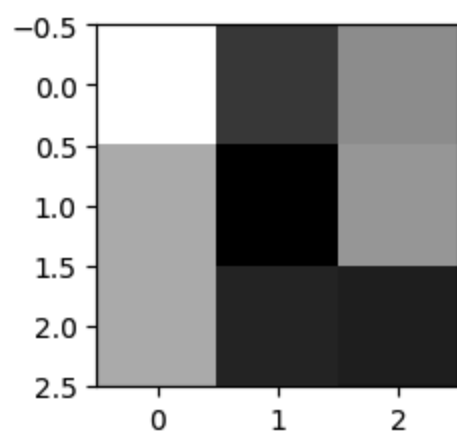
Weights of conv2d_5

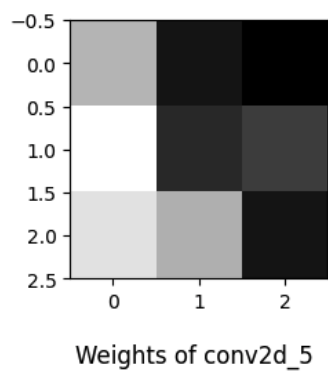
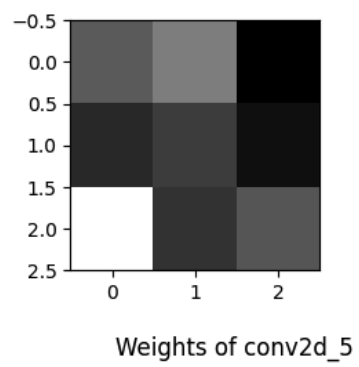


Weights of conv2d_5



Weights of conv2d_5



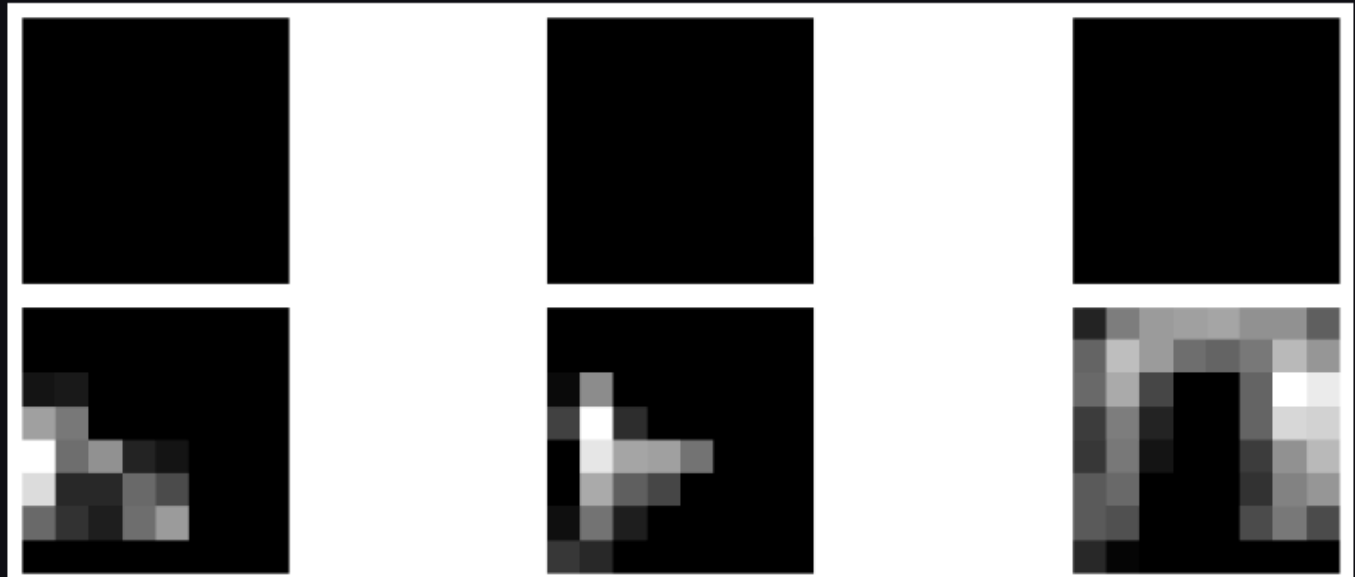


```
2026-09-04 17:16:55.282155: W tensorflow/core/framework/local_rendevous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
--- First layer ---
Data type:float32
Feature map shape: (1, 60, 60, 32)
Memory:460800 bytes
```



Visualization of Early Feature Map

```
--- Last Layer ---
Data type:float32
Feature map shape: (1, 8, 8, 128)
Memory:32768 bytes
```



Visualization of Later Feature Map

How do feature maps change as they progress through the models?

As we progress through the model, our feature maps are decreasing in discernible visual detail, with only major parts of the image staying to the later layers. We go from spatially large images

to spatially small due to the convolutions. The high level details we can visually see in the first layer are slowly melted away as we decrease in resolution.

How does this differ from weights?

Feature maps are input-dependent activations produced by each layer, while weights are learned parameters that remain fixed during inference and are applied to every input. As depth increases, feature maps generally become spatially smaller and represent more abstract, class-related features. The size of the kernel weights depends mainly on the kernel dimensions and the number of input and output channels.

Does depth correlate with feature maps vs kernels differently?

Feature maps correlate differently than kernels when it comes to depth. Feature maps correlate with depth when it comes to shrinking spatial size of images. When the image keeps going through the CNN, it loses visual data and gains a larger output that is not visually distinct. Weights correlate when it comes to interpretability. Images are altered in a way that edges are prioritized during computation, and then the edge is blurred during computation, leading to a “loss” of visual appearance.

Export the weights, biases of the model and the intermediate feature maps of any three validation set inputs as binary file

```

pathlib.Path(model_dir).mkdir(exist_ok=True)

# Export each image
conv_index = dense_index = 1 # layer index starts from one
for layer_idx, layer in enumerate(model.layers):
    if re.match(r'(conv|dense)', layer.name):
        weight_file_name = os.path.join(model_dir, f'{layer.name}_weights.bin')
        bias_file_name = os.path.join(model_dir, f'{layer.name}_bias.bin')
    else: continue

    assert layer.weights[0].name.endswith('kernel')
    assert layer.weights[1].name.endswith('bias')

    # TODO: Your Code Here

    kernel, bias = layer.get_weights()

    kernel.astype(np.float32).tofile(weight_file_name)
    bias.astype(np.float32).tofile(bias_file_name)

    print(f'Exported {layer.name}: kernel {kernel.shape}')
    print(f'{weight_file_name}, bias {bias.shape} {bias_file_name}')

Exported conv2d: kernel (5, 5, 3, 32)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_weights.bin, bias (32,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_bias.bin
Exported conv2d 1: kernel (5, 5, 32, 32)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_1_weights.bin, bias (32,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_1_bias.bin
Exported conv2d 2: kernel (3, 3, 32, 64)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_2_weights.bin, bias (64,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_2_bias.bin
Exported conv2d 3: kernel (3, 3, 64, 64)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_3_weights.bin, bias (64,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_3_bias.bin
Exported conv2d 4: kernel (3, 3, 64, 64)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_4_weights.bin, bias (64,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_4_bias.bin
Exported conv2d 5: kernel (3, 3, 64, 128)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_5_weights.bin, bias (128,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/conv2d_5_bias.bin
Exported dense: kernel (2048, 256)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/dense_weights.bin, bias (256,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/dense_bias.bin
Exported dense 1: kernel (256, 200)
/home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/dense_1_weights.bin, bias (200,) /home/jwk0425/cpre_487/lab1/cpre-487-587-lab1/model_data/dense_1_bias.bin

```

Exporting of weights

[Model Analytics/Metrics with Tensorboard]

Report the end-to-end latencies of your three test images

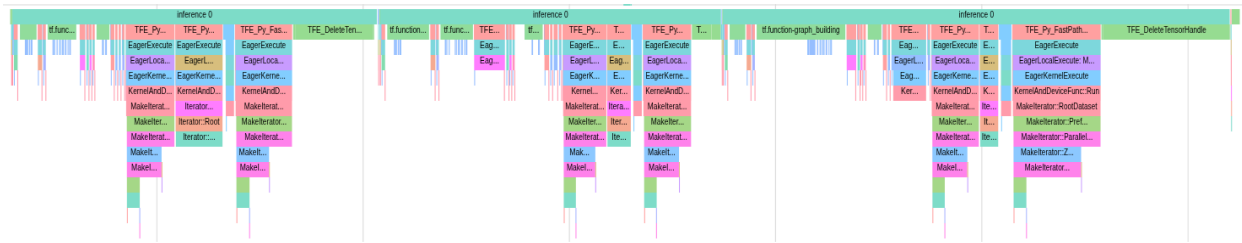
```

Image 0 latency : 66.444 ms
Image 1 latency : 62.231 ms
Image 2 latency : 63.018 ms

```

Image latencies

Report which layer type contributes most to the overall inference computation latency

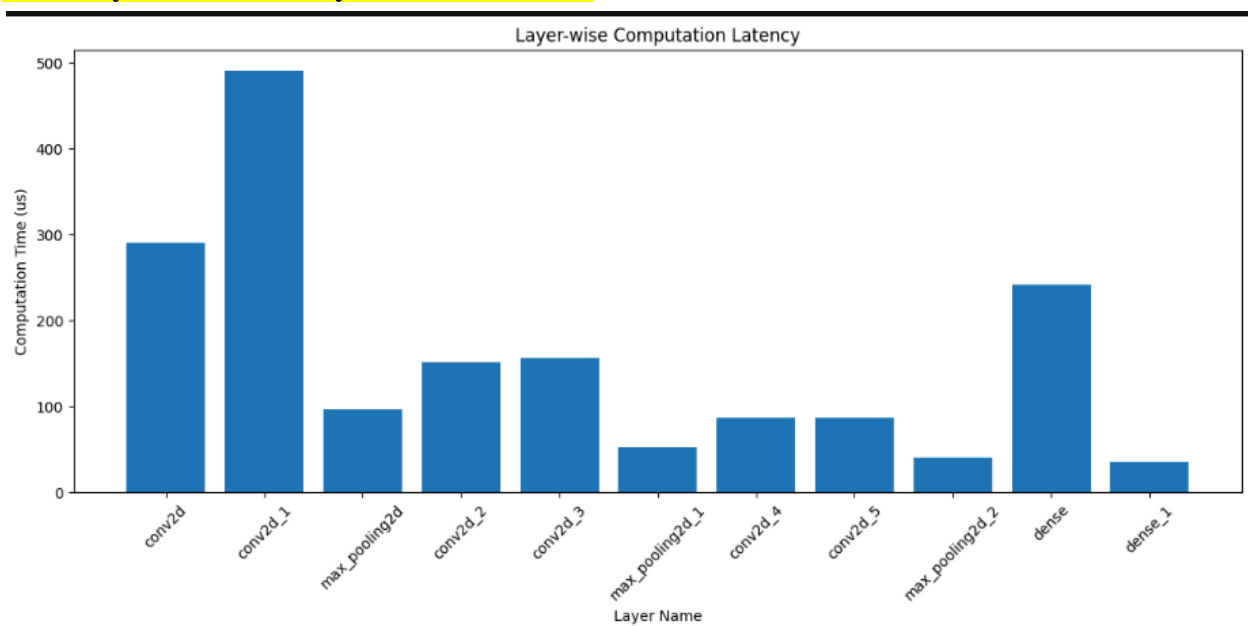


The convolutional layers are the most intensive when it comes to computational latency.

State a brief reason for the observed behaviour. Report the layer in the model that dominates the computation latency and why?

The convolutional layers contribute the most to the overall computation latency because convolution requires many multiply-and-accumulate operations for each output element. In particular, conv2d_1 has the highest computation latency in this model. This layer operates on a relatively large spatial feature map while also processing 32 input channels, resulting in substantially more convolution operations than the first convolutional layer.

For any of the chosen inputs, plot the layer-wise computation latency with the computation time on y-axis and the layer name on x-axis



For our model, conv2d 1 has the largest computation latency

Perform online inference for a set of 10, 100, and 1,000 images. Report the end-to-end latency and throughput for each

```
10 images:  
End-to-end latency: 0.658 s  
Throughput: 15.204 images/s
```

10 image inference

```
100 images:  
End-to-end latency: 5.680 s  
Throughput: 17.605 images/s
```

100 image inference

```
1000 images:  
End-to-end latency: 54.164 s  
Throughput: 18.463 images/s
```

1000 image inference

Perform batch inference for 1,000 of the validation set images using batch sizes of 20, 40, 100 and 200 images. Report the end-to-end latency and throughput for each. Comparing it with online inference of 1000 image

```
Batch size 20:  
End-to-end latency: 4.201 s  
Throughput: 238.016 images/s
```

20 Batch size

```
Batch size 40:  
End-to-end latency: 3.577 s  
Throughput: 279.603 images/s
```

40 Batch size

```
Batch size 100:  
End-to-end latency: 1.654 s  
Throughput: 604.422 images/s  
Batch size 200:  
End-to-end latency: 0.436 s  
Throughput: 2295.090 images/s
```

100 and 200 Batch size

Method	Total Images	End-toEnd latency	Throughput
Online	1000	54.164s	18.463
Batch=20	1000	4.201s	238.016
Batch=40	1000	3.577s	279.603
Batch=100	1000	1.654s	604.422
Batch=200	1000	0.436s	2295.09

Image comparison for Online vs Batch

Report the differences you observed in the end-to-end latencies for various batch size

As the batch size increased, the end-to-end latency for processing the 1,000 images generally decreased. The batch size of 20 required approximately 4.2 seconds, while larger batch sizes further reduced the total processing time.

Write briefly about the possible reasons for the observed behaviour. Describe when one method would be preferred than the other and visa versa

Batch inference can reduce the total latency by processing multiple images together and reducing the overhead associated with individual inference calls. In a GPU environment, batching can also take advantage of parallel computation. In our experiment, the observed improvement should primarily be interpreted in the context of the hardware and TensorFlow execution environment used for profiling.

Online inference is more appropriate when individual predictions must be processed immediately, such as real-time applications where each image arrives independently. Batch inference is more appropriate when a large number of images are available at the same time because multiple inputs can be processed together, reducing the overhead per image and generally improving throughput.

Identify the kernels most critical for optimization. Why? Does the computational expense for each layer operation make sense (Consider what each layer operation is doing and how that correlates to the latency it incurs in both single/online and batch inference)?

The convolutional kernels are the most critical for optimization because the convolutional layers account for a large portion of the computation. In particular, conv2d_1 is a key target because it operates on a relatively large spatial feature map with 32 input channels.

This behavior is consistent with the computational structure of the model. Earlier convolutional layers operate on larger spatial dimensions, so they can require significant computation even when the number of channels is smaller. As the spatial dimensions decrease through pooling, later convolutional layers operate on smaller feature maps, although some have more channels. The observed latency therefore depends on both spatial dimensions and the number of input and output channels rather than depth alone.

[Training]

Use three different batch sizes(e.g., 2,4,8,or 16). Report the best Top-1 and Top-5 validation set accuracy.

```
Batch Size: 32
Best Top-1 Validation Accuracy: 0.50%
Best Top-5 Validation Accuracy: 2.50%
```

```
Batch Size: 64
Best Top-1 Validation Accuracy: 19.13%
Best Top-5 Validation Accuracy: 42.56%
```

```
Batch Size: 128
Best Top-1 Validation Accuracy: 22.41%
Best Top-5 Validation Accuracy: 46.96%
```

Report how the number of epochs used in training impacts the validation accuracy.

```
Epochs: 3
Best Top-1 Validation Accuracy: 13.09%
Best Top-5 Validation Accuracy: 33.20%
```

```
Epochs: 10
Best Top-1 Validation Accuracy: 0.50%
Best Top-5 Validation Accuracy: 2.50%
```

```
Epochs: 100
Best Top-1 Validation Accuracy: 0.50%
Best Top-5 Validation Accuracy: 2.50%
```

In our experiments, the 3-epoch training run achieved the highest validation accuracy, reaching 13.09% Top-1 and 33.20% Top-5 accuracy. The 10-epoch and 100-epoch runs remained at approximately 0.50% Top-1 and 2.50% Top-5 accuracy. Therefore, simply increasing the number of training epochs did not improve the accuracy in these runs.

Report the minimum number of training epochs needed to achieve the best possible validation set accuracy for all training runs.

In our experiments, the 3-epoch training run achieved the highest validation accuracy, reaching 13.09% Top-1 and 33.20% Top-5 accuracy. The 10-epoch and 100-epoch runs remained at approximately 0.50% Top-1 and 2.50% Top-5 accuracy. Their loss and accuracy showed little improvement during training, indicating that these runs did not converge successfully. Therefore, increasing the number of epochs alone did not improve validation accuracy under these training conditions.

What happens to validation accuracy when you train to a large number of epochs? State possible reason about the observed behaviour in the training file template.

When the model was trained for 10 and 100 epochs, its validation accuracy remained near 0.50% Top-1 and 2.50% Top-5 instead of improving. Because the training loss and accuracy also changed very little, the model likely failed to converge during these runs. One possible reason is that the training configuration, such as the learning rate or optimizer settings, prevented the model weights from being updated effectively.